RETAIL SALES & CUSTOMER BEHAVIOUR ANALYTICS

Student Project Assignment — Questions (Revised)

Instructions: Use the six supplied CSV files. Perform data exploration, cleaning, filtering, aggregation, pivot-table analysis, joins, and visualisation using Pandas, Matplotlib, and Seaborn. Preserve the original DataFrames and explain important cleaning decisions.

PART 1 — Data Understanding & Exploration

1. Load all six CSV files into separate Pandas DataFrames.

2. Display the shape (number of rows and columns) of every DataFrame.

3. Display the first 10 rows of each DataFrame using `head()`.

4. Display the last 5 rows of each DataFrame using `tail()`.

5. Display 5 random rows from each DataFrame using `sample()`.

6. Inspect the data types and non-null counts using `info()`.

7. Generate statistical summaries using `describe()`.

8. Find the number of unique values in each column.

9. Display value counts for categorical columns such as order_status, sales_channel, payment_method, customer_segment, category, brand, and payment_status.

10. Check whether date columns are stored as datetime. Convert order_date, signup_date, and payment_date where necessary.

PART 2 — Data Quality Audit & Cleaning

1. Calculate the total number of missing values in every column of every table.

2. Identify duplicate rows in the customers table.

3. Find duplicate customer_id values and decide how to handle the duplicated customer record.

4. Inspect missing values in age, city, preferred_contact, shipping_type, discount_pct, payment_status, payment_date, and return_reason.

5. Handle missing values in numeric columns (such as age and discount_pct) by filling them with the mean or median. Choose a suitable method for each column.

6. Handle missing values in text/categorical columns (such as city, preferred_contact, shipping_type, payment_status, and return_reason) by filling them with a suitable string value, such as 'Unknown'.

7. For rows with missing payment_date, decide whether to keep or remove them. Use `dropna()` where appropriate and explain your choice.

8. After handling missing values, check the missing-value count again using `isnull().sum()`.

9. Identify product_id values in order_items that do not exist in products. Use a suitable Pandas approach such as `isin()` or a merge.

PART 3 — Data Filtering

1. Display all orders placed through the Mobile App.

2. Filter orders with Delivered status that were placed in 2026.

3. Find orders that use Express shipping and UPI as the payment method.

4. Find customers aged between 25 and 40 who are from Rajasthan.

5. Find products whose unit_price is greater than the average product price.

6. Find order items where quantity is at least 3 and discount_pct is at least 10%.

7. Filter payments whose status is Failed or Pending. Which payment mode has the most such records?

8. Find all Returned orders and display their return reasons and refund statuses.

9. Find the 10 order items with the highest line_total.

10. Find customers who placed orders in both 2025 and 2026.

PART 4 — Grouping, Aggregation & Pivot Tables

1. Calculate the number of orders for each order_status.

2. Calculate the number of orders for each sales_channel.

3. Calculate the average customer age for each customer_segment.

4. Calculate the total quantity sold for each product category.

5. Calculate total revenue for each product category.

6. Calculate revenue by brand and identify the top five brands.

7. Calculate monthly order count and monthly revenue.

8. Calculate Average Order Value (AOV) for each sales channel. Clearly define which order statuses are included.

9. Use multiple aggregations to calculate order count, total revenue, average discount, and total quantity by category.

10. Calculate the number of customers in each city.

11. Calculate total revenue generated by each city.

12. Identify the city that generates the highest revenue.

13. Create a pivot table with category as the index, sales_channel as columns, and line_total as values using sum as the aggregation function.

14. Create a pivot table with month as the index, order_status as columns, and order count as values.

15. Create a pivot table showing average sales by customer_segment and category.

16. Create a pivot table showing revenue by category and brand.

PART 5 — Pandas Joins & Merging

1. Merge orders with customers using customer_id. Use a left join to attach customer details to every order.

2. Merge order_items with products using product_id. Use an inner join and inspect unmatched product IDs separately.

3. Create a detailed sales DataFrame by merging orders, order_items, and products.

4. Add customer information to the detailed sales DataFrame.

5. Merge orders with payments to analyse payment status against order status.

6. Merge returns with orders to retrieve complete information about returned orders.

7. Merge returns, orders, order_items, and products to identify which product categories have the most returns.

8. Compare the row counts produced by inner and left joins when merging orders with customers. Explain any difference.

9. Use `merge(indicator=True)` to identify matched and unmatched records.

PART 6 — Matplotlib Visualisation Tasks

1. Create a histogram showing the distribution of customer ages. Add a title, axis labels, bins, and grid.

2. Create a scatter plot showing the relationship between quantity and line_total. Use colour or grouping by category if useful.

3. Create a line plot showing monthly revenue over time. Add markers and readable date labels.

4. Create another line plot comparing monthly revenue for at least three product categories.

PART 7 — Seaborn Visualisation Tasks

1. Create a bar plot comparing total revenue by product category. Sort categories by revenue.

2. Create a box plot comparing line_total across product categories. Identify possible outliers.

3. Create a correlation heatmap for suitable numerical columns, such as quantity, unit_price, discount_pct, line_total, and unit_cost. Add annotations.

4. Create a KDE plot showing the distribution of line_total. Optionally compare categories using hue.

5. Create a violin plot comparing line_total distributions across sales channels or product categories.

6. Create a swarm plot comparing line_total across categories. Use a manageable sample if the plot becomes crowded.

7. Create a strip plot showing individual line_total observations by category. Use jitter and transparency where appropriate.

SUBMISSION REQUIREMENTS

Submit a well-organised Jupyter Notebook (.ipynb) with each question followed by its solution. Include the required Pandas outputs, cleaned or merged DataFrames where relevant, all requested plots, and a short interpretation below every chart.

SUGGESTED EVALUATION RUBRIC — 100 MARKS

Section	Marks
Data exploration	10
Data cleaning	20
Filtering, GroupBy & Pivot Tables	20
Pandas Joins	20
Matplotlib & Seaborn	30
Total	100